Recruiterflow

Internship Take-Home Assignments

QA Automation · Backend · Frontend · Full Stack

How to use this document

This pack contains all three internship take-home assignments. Find the role you are applying for in the table below and complete **only the section(s) listed for that role**. Each section is the complete, original assignment brief, including its own instructions, timelines and submission details.

Applying for	Complete	Assignment
QA Automation Intern	Section A only	QA Automation Take-Home — Playwright + TypeScript (UI + API tests)
Backend Developer Intern	Section B only	Build-a-Recruiter-Bot — backend service + SQL
Frontend Developer Intern	Section C only	Trip Splitter — React, Redux Toolkit, strict TypeScript
Full Stack Developer Intern	**All three: A, B and C**	Every assignment in this pack

Full Stack applicants

Please attempt all three assignments. You may submit them as separate repositories or as one repository with a clear folder per section (e.g. `qa/`, `backend/`, `frontend/`), each with its own README. Partial work is fine — say in the README what is missing and what you would do next.

Contents

Section	Role	Pages
Section A	QA Automation Intern	3–6
Section B	Backend Developer Intern	7–14
Section C	Frontend Developer Intern	15–19

Each section begins with the original assignment brief. Read the whole section before starting — submission windows and deliverables differ between roles.

QA Automation Take-Home — Freshers

Playwright + TypeScript • Submission window: 48 hours • Suggested effort: 2–3 hours • Deliverable: GitHub repository link

Overview

Thank you for your interest in joining our QA team. This assignment helps us understand how you think about testing and how comfortable you are writing basic automated tests. We do not expect you to have used Playwright before. Reading the official docs and learning as you go is completely fine, and it is part of what we want to see.

You have **48 hours** from the time you receive this email to submit. The tests are intentionally simple, and we expect about 2–3 hours of actual work, including setup. If something does not work, submit what you have and explain what you tried. A clear explanation of a failing test is worth more to us than a test you do not understand.

What to Build

A small Playwright + TypeScript test suite with two parts: UI tests against a demo shop and API tests against a demo REST API. Both sites are public, free and need no sign-up.

Part 1 — UI Automation

Target site: https://www.saucedemo.com

Test users (the full list is shown on the login page):

- `standard_user` / `secret_sauce`
- `locked_out_user` / `secret_sauce`

Write tests for these scenarios:

1. **Valid login:** log in as `standard_user` and verify the "Products" title is visible.
2. **Locked-out user:** log in as `locked_out_user` and verify an error message containing "locked out" is shown.
3. **Add to cart:** log in, click "Add to cart" on any one product, and verify the cart badge shows **1**.

Part 2 — API Automation

Target API: https://reqres.in

Use the public endpoints documented on the site.

Note: ReqRes currently requires an API key for API requests. Create a free ReqRes account and generate an API key. Pass the key in the x-api-key request header for the API tests.

Use Playwright's built-in `request` fixture for these tests. You do not need a browser for API tests.

4. **Get a user:** `GET /api/users/2`. Verify the status is **200** and the response has a `data.email` value.
5. **Create a user:** `POST /api/users` with the JSON body `{ "name": "morpheus", "job": "leader" }`. Verify the status is **201** and the response `name` is "morpheus".

Tip: read the reqres.in homepage carefully before you start. If a request returns 401, the site's documentation explains what header to add.

Bonus (only if you have time)

These are optional. Skipping them will not count against you.

- Remove the product from the cart and verify the badge disappears.
- Log out from the side menu and verify you are back on the login page.
- `GET /api/users/23` and verify the status is **404**.
- Move the login steps into a small helper or Page Object (e.g. `pages/LoginPage.ts`) so tests don't repeat them.

Getting Started

1. Install Node.js (version 18 or newer).
2. Create a new folder and run: `npm init playwright@latest`
3. Choose **TypeScript**, keep the default `tests` folder, and say **yes** to installing browsers.
4. Run the example test with `npx playwright test` to confirm everything works, then delete it and add your own.

Useful docs:

- Writing tests: https://playwright.dev/docs/writing-tests
- Locators: https://playwright.dev/docs/locators
- API testing: https://playwright.dev/docs/api-testing

Suggested Folder Structure

```
my-qa-assignment/
├── tests/
│   ├── ui/
│   │   ├── login.spec.ts
│   │   └── cart.spec.ts
│   └── api/
│       └── users.spec.ts
├── pages/            (optional, bonus)
├── playwright.config.ts
├── package.json
└── README.md
```

What We Are Looking For

- **Tests that run.** We should be able to clone your repo, run `npm install` and `npx playwright test`, and see results.

- **Clear test names**, e.g. `test('locked out user sees an error', ...)`.
- **Simple locators.** Prefer `getByRole`, `getByText` or `getByTestId` over long CSS or XPath. (saucedemo uses `data-test` attributes; set `testIdAttribute: 'data-test'` in `playwright.config.ts` to use them with `getByTestId`.)
- **Real checks.** Every test should end with at least one `expect(...)`, and every Playwright call should be awaited.
- **A short README** saying how to install and run the tests, plus **a few commits** instead of one.

What You Do NOT Need to Do

- Run tests on multiple browsers (Chromium only is fine).
- Set up CI, Docker or reporting tools.
- Build a full framework or cover every edge case.
- Finish every scenario. Partial work with a clear explanation is acceptable.

How to Submit

1. Push your code to a **public** GitHub repository.
2. Check that the README explains how to install and run the tests.
3. Reply to the recruiter email with the GitHub link within 48 hours.
4. In the same reply, write **3–5 sentences** covering: what you completed, anything you got stuck on and how you tried to solve it, and what you learned or would improve with more time.

How We Will Evaluate

An engineer from our team will review your repository. In the next round we will ask you to walk us through your code and make a small change live, so make sure you understand everything you submit. Using AI tools or online examples is allowed, but you should be able to explain every line.

Area	Weight
Scenario coverage (tests exist and run)	30%
Code readability and test naming	20%
Locator and assertion quality	20%
README and Git history	15%
Written reflection in your email	15%

If anything is unclear, email us before you start. We would rather answer a question than have you guess. Good luck!

Internal notes for reviewers (remove before sending)

Key differences from the experienced version:

- 48-hour window instead of a strict 2-hour time-box; about 2–3 hours of work.
- 5 core tests (3 UI, 2 API), each a single action plus one or two checks. No checkout, sorting, or multi-item cart.
- Page Object Model, the negative API case, remove from cart, and logout are bonus only.
- Includes setup steps, doc links, and locator hints, since freshers may not have used Playwright.
- Scoring puts more weight on "does it run" and on the reflection (learning ability), less on architecture.
- The next round includes a small live change, to check the candidate understands their own code.

Heads-up on reqres.in: reqres now requires a free API key sent as an `x-api-key` header, or requests return 401. Verify the current requirement on reqres.in before sending, and decide whether to state the header in the brief or leave it as a small "read the docs" test (the current wording does the latter).

Build-a-Recruiter-Bot

Backend Developer Assignment — RecruiterFlow Engineering

Welcome

Hi! Thanks for taking the time to work through this. We built this assignment to feel like an actual day of backend work at RecruiterFlow, not a whiteboard trivia contest — so expect some silliness in the data and real design decisions in the work.
There's no single "correct" solution. We care much more about **how you think and why you built it the way you did** than about matching some hidden reference implementation. You'll get a chance to walk us through your choices afterward, so keep notes on the trade-offs you made as you go.

The Scenario

RecruiterFlow is prototyping **Recruiter Bot** — a small backend service that ingests candidate profiles and job openings, then ranks the best matches for each. To keep grading (and building) fun, our test data isn't real people — it's a cast of well-known fictional characters, each with invented skills and quirks. Somewhere out there, Sherlock Holmes is trying to get a job as a backend detective, and Michael Scott is applying to be a sales engineer.
Your job: build the backend that makes those matches happen.

Part 1 — The Recruiter Bot (core project)

Seed data: Candidates

Name	Skills	Experience (yrs)	Availability	Traits	Quirk
Sherlock H.	deduction, pattern-recognition, forensics	8	Immediate	analytical, blunt	Solves problems by eliminating the impossible; occasionally insufferable in standups.
Hermione G.	research, time-management, public-speaki	4	2 weeks	detail-oriented, overachiever	Reads the entire documentation before writing a

Name	Skills	Experience (yrs)	Availability	Traits	Quirk
	ng				single line of code.
Tony S.	systems-design, rapid-prototyping, leadership	12	Not looking	confident, innovative	Ships an MVP overnight, refuses to write tests.
Leslie K.	project-management, stakeholder-management, public-speaking	6	Immediate	tenacious, organized	Has a binder for everything, including the binder.
Ron S.	woodworking, minimalism, negotiation	15	Not looking	stubborn, principled	Refuses to use more than one monitor.
Rick S.	systems-design, rapid-prototyping, chemistry	20	Immediate	genius, reckless	Solution works, mechanism deeply concerning.
Elle W.	persuasion, research, public-speaking	3	Immediate	optimistic, sharp	Underestimated in every standup, correct in every retro.
MacGyver	rapid-prototyping, resourcefulness, chemistry, systems-design	10	2 weeks	calm, improviser	Fixes production outages with duct tape and a paperclip metaphor.
Sheldon C.	theoretical-analysis, pattern-recog	9	Immediate	rigid, brilliant	Correct 95% of the time, insufferable

Name	Skills	Experience (yrs)	Availability	Traits	Quirk
	nition, research				100% of the time.
Katniss E.	precision, strategy, crisis-management	5	Immediate	resilient, decisive	Excellent under pressure, terrible with public speaking.
Michael S.	sales, public-speaking, team-building	11	Not looking	enthusiastic, chaotic	World's best boss, according to a mug he bought himself.
Olivia P.	crisis-management, negotiation, strategy, leadership	13	2 weeks	decisive, intense	Handles it. Whatever it is.
Ted L.	team-building, optimism, mentorship, public-speaking	7	Immediate	empathetic, persistent	Turns every technical setback into a folksy metaphor.
Miranda P.	leadership, negotiation, stakeholder-management, precision	18	Not looking	demanding, decisive	Reviews every PR personally. Says nothing. Everyone panics.
Dwight S.	sales, negotiation, security, loyalty	9	Immediate	intense, loyal	Assistant to the regional backend engineer.

Seed data: Jobs

Title	Required Skills	Min Experience	Culture Keywords	Tagline
Backend Detective	deduction, pattern-recognition, forensics	3	analytical, autonomous	We have a bug. We have no leads. We have you.
Rapid Prototyping Engineer	rapid-prototyping, systems-design	2	innovative, fast-paced	Ship first, document never (kidding — please document).
Developer Relations Lead	public-speaking, research	2	energetic, curious	Explain complex things to confused humans, cheerfully.
Engineering Manager, Chaos Team	team-building, stakeholder-management, leadership	5	empathetic, organized	Herd cats. The cats are senior engineers.
Incident Commander	crisis-management, strategy, negotiation	4	decisive, calm-under-pressure	3am page. You're the one who picks up.
Sales Engineer	sales, public-speaking, negotiation	3	enthusiastic, persistent	Sell the vision, then go build it.

What to build

1. **Ingestion**: a way to load the candidates and jobs above into your own data store (an API endpoint, a seed script — your call).
2. **Your own schema**: design the database schema for candidates, jobs, and however you choose to represent a "match." **Raw SQL only** — no ORM, query builder, or anything that generates SQL for you. Hand-written DDL and hand-written queries, full stop.
3. **Matching API**, at minimum:
    - GET /jobs/{id}/matches — ranked candidates for a job, with a score and a short reason.
    - GET /candidates/{id}/matches — ranked jobs for a candidate (the reverse direction).

4. **The matching algorithm** — this is the heart of the assignment. It *must* account for:
    - Skill overlap between the candidate and the job's required skills.
    - Experience fit against the job's minimum experience.

Beyond those two, the design is entirely yours: how you weight them, whether/how you use traits, culture keywords, availability, tie-breaking — all up to you. Be ready to explain and defend whatever you chose.

Constraints

- **Any programming language**. Use whatever you're strongest in.
- **Any relational database** (SQLite, MySQL, Postgres, etc.), but interact with it via **raw, hand-written SQL** — no ORMs or query-builder abstractions.
- No frontend needed. A README with curl examples, or a Postman collection, is enough to demonstrate the API works.
- Tests and API docs are **not required**, but are a nice bonus if you have time left over.

Part 2 — The SQL Detective Challenge

Separately from your project's own schema, here's a small, slightly messy "hiring ops" dataset. Load it as-is (don't redesign it) and write the SQL to answer the questions below. This part is about your raw querying skill, independent of your Part 1 code.

Schema

```
CREATE TABLE recruiters (
  id INT PRIMARY KEY,
  name VARCHAR(100),
  region VARCHAR(50)
);

CREATE TABLE job_postings (
  id INT PRIMARY KEY,
  title VARCHAR(100),
  recruiter_id INT,
  department VARCHAR(50),
  opened_date DATE,
  status VARCHAR(20) -- 'open', 'closed', 'on_hold'
);

CREATE TABLE applicants (
  id INT PRIMARY KEY,
  full_name VARCHAR(100),
  email VARCHAR(100),
  source VARCHAR(50), -- nullable
  applied_date DATE
);
```

```
CREATE TABLE interviews (
  id INT PRIMARY KEY,
  applicant_id INT,
  job_posting_id INT,
  stage VARCHAR(20),      -- 'Screen', 'Technical', 'Final'
  scheduled_date DATE,
  outcome VARCHAR(20)     -- 'passed', 'failed', 'no_show', NULL (pending)
);
```

Seed data

```
INSERT INTO recruiters VALUES
(1, 'Priya Shah', 'APAC'),
(2, 'Daniel Ortiz', 'NA'),
(3, 'Fatima Al-Sayed', 'EMEA'),
(4, 'Wei Zhang', 'APAC'),
(5, 'Grace Okafor', 'EMEA');

INSERT INTO job_postings VALUES
(1, 'Backend Engineer I', 1, 'Engineering', '2026-01-05', 'open'),
(2, 'Backend Engineer I', 2, 'Engineering', '2026-01-10', 'open'),
(3, 'Data Analyst', 3, 'Data', '2026-01-08', 'closed'),
(4, 'Backend Engineer II', 1, 'Engineering', '2026-02-01', 'open'),
(5, 'DevOps Engineer', 4, 'Infrastructure', '2026-01-20', 'open'),
(6, 'Data Analyst', 5, 'Data', '2026-02-05', 'on_hold'),
(7, 'Backend Engineer I', 2, 'Engineering', '2026-02-15', 'closed');

INSERT INTO applicants VALUES
(1, 'Ananya Rao', 'ananya.rao@mail.com', 'LinkedIn', '2026-01-06'),
(2, 'Ananya Rao', 'Ananya.Rao@mail.com', 'Referral', '2026-02-04'),
(3, 'Ben Turner', 'ben.turner@mail.com', 'Job Board', '2026-01-07'),
(4, 'Chloe Martin', 'chloe.martin@mail.com', NULL, '2026-01-09'),
(5, 'Devansh Patel', 'devansh.patel@mail.com', 'LinkedIn', '2026-01-11'),
(6, 'Elena Popescu', 'elena.popescu@mail.com', 'Referral', '2026-01-12'),
(7, 'Farid Haidari', 'farid.haidari@mail.com', 'Job Board', '2026-01-13'),
(8, 'Grace Lin', 'grace.lin@mail.com', 'LinkedIn', '2026-01-15'),
(9, 'Hassan Malik', 'hassan.malik@mail.com', NULL, '2026-01-16'),
(10, 'Isla Fraser', 'isla.fraser@mail.com', 'Referral', '2026-01-18'),
(11, 'Jonas Weber', 'jonas.weber@mail.com', 'Job Board', '2026-02-10');

INSERT INTO interviews VALUES
(1, 1, 1, 'Screen', '2026-01-10', 'passed'),
(2, 1, 1, 'Technical', '2026-01-15', 'passed'),
(3, 1, 1, 'Final', '2026-01-22', 'passed'),
(4, 2, 4, 'Screen', '2026-02-05', 'passed'),
(5, 3, 1, 'Screen', '2026-01-11', 'passed'),
(6, 3, 1, 'Technical', '2026-01-16', 'failed'),
```

```
(7, 3, 3, 'Final', '2026-01-28', 'passed'),
(8, 4, 1, 'Screen', '2026-01-12', 'passed'),
(9, 4, 1, 'Final', '2026-01-24', 'failed'),
(10, 5, 2, 'Screen', '2026-01-14', 'passed'),
(11, 5, 2, 'Technical', '2026-01-19', 'passed'),
(12, 5, 2, 'Final', '2026-01-25', 'passed'),
(13, 6, 2, 'Screen', '2026-01-15', 'passed'),
(14, 6, 2, 'Final', '2026-01-26', 'failed'),
(15, 7, 2, 'Screen', '2026-01-16', 'no_show'),
(16, 8, 4, 'Screen', '2026-02-03', 'passed'),
(17, 8, 4, 'Technical', '2026-02-08', 'passed'),
(18, 8, 4, 'Final', '2026-02-12', 'passed'),
(19, 9, 4, 'Screen', '2026-02-04', 'passed'),
(20, 9, 4, 'Final', '2026-02-14', 'passed'),
(21, 10, 5, 'Screen', '2026-01-22', 'passed'),
(22, 10, 5, 'Technical', '2026-01-28', NULL),
(23, 11, 7, 'Final', '2026-02-20', 'passed'),
(24, 6, 7, 'Final', '2026-02-21', 'failed');
```

Questions

1. List all currently **open** job postings, along with the name of the recruiter who owns each one.
2. For each job posting, how many applicants reached the **Final** interview stage?
3. Find every person who effectively applied to **more than one** job posting. Careful — the data isn't as clean as it looks.
4. For each recruiter, compute their **Final-stage conversion rate** (passed ÷ total Final-stage interviews), but only include recruiters with at least 3 Final-stage interviews.
5. *(Bonus)* Using a window function, find the recruiter with the most successful placements (passed at Final stage) **per department**.

Submit your answers as a single .sql file (e.g. sql_detective.sql), one query per question, each preceded by a comment noting the question number.

What to Submit

- A link to a Git repository (public, or private with access granted) containing your Part 1 project.
- A **README** in that repo covering:
  - How to install and run it locally.
  - A short (few-paragraph) write-up of the decisions and trade-offs you made — especially in the matching algorithm.
  - A note on where, if anywhere, you used AI tools (ChatGPT, Copilot, etc.) while working on this. This isn't a penalty — we're just curious how you use them, and it helps guide our conversation afterward.
- Your sql_detective.sql file from Part 2, included in the same repo.

Time & AI Tools

Budget roughly **4 hours** total (suggested split: ~2.5–3 hours on Part 1, ~45 min–1 hour on Part 2, the rest on your README). You'll have a few days of calendar time to fit that in around everything else going on in your life — there's no need to do it all in one sitting.
You're welcome to use AI coding assistants. We're not trying to catch you using them — we're trying to have a good conversation afterward about what you built and why, and that conversation works best when it's genuinely your understanding on display.

What Happens Next

After you submit, we'll set up a ~30–45 minute conversation where you walk us through your solution, we ask about a few of your decisions, and we throw a couple of "what if" curveballs your way. Come as you are — there's no additional prep needed beyond understanding what you built.

Questions?

If anything here is ambiguous, that's partly on purpose — recruiting requirements in the real world are rarely fully specified either, and we're curious how you handle that. But if you're genuinely stuck on something outside the spirit of the exercise, reach out and ask.
Good luck, and have fun with it — Sherlock is counting on you.

FRONTEND DEVELOPER INTERN ASSIGNMENT

Fresher Take-Home: Trip Splitter

The short version

You will build a small two-page app with React, Redux Toolkit and strict TypeScript, that keeps its data in the browser.

We care more about clean, correct, well-typed code than about how much you build. Unfinished is fine if your README says what is missing.

The problem

Five friends just got back from a trip to Goa. Different people paid for different things along the way: the villa, dinners, a scuba session only three of them joined, the cab from the airport. Now nobody is sure who owes whom.

Build a small app where the group records shared expenses, chooses how each one is split, and sees every person's balance: how much they get back, or how much they owe.

What to build

Navbar

- Links to **Expenses** (`/`) and **Add expense** (`/expenses/new`). The current page is visibly marked.
- Any other URL shows a simple "Page not found" with a link back to Expenses.

Add expense

A form with these fields:

- **Description** (text)
- **Amount** in ₹, up to 2 decimals
- **Paid by**, chosen from the trip members
- **Date**
- **Category**: Food, Travel, Stay, Activities or Other
- **Split**, one of three types:
    - **Equally** between the members the user selects
    - **Exact amounts** in ₹ for each member
    - **Percentages** for each member

Save adds the expense, then takes the user to Expenses with a message that it was added. **Cancel** goes back without saving.

Expenses

- A **balance for each member**: "gets back ₹X", "owes ₹X" or "settled up".
- A **table of expenses**, newest date first, with these columns: Date, Description, Category, Paid by, Amount, and Split (for example "Equally, 4 people").

- A **category filter** for the table, and the total of the expenses currently shown.
- **Delete** an expense after the user confirms. Balances update straight away.
- A clear **empty state** when there are no expenses, or none in the chosen category.

Rules the app must follow

Validation

When the user presses Save, show an error next to every field that breaks a rule, and do not save anything.

- **Description** is required, at most 60 characters.
- **Amount** is required, more than ₹0 and at most ₹10,00,000, with no more than 2 decimals. A negative amount shows "Amount must be more than ₹0".
- **Paid by**, **date** and **category** are required. The date cannot be in the future.
- **Equally:** at least one member is selected.
- **Exact amounts:** each is ₹0 or more, and together they equal the expense amount exactly. Otherwise show how much is left or over, for example "₹12.50 left to assign".
- **Percentages:** each is between 0 and 100 with up to 2 decimals, and together they equal exactly 100.

Money

- A member's **balance** is what they paid, minus their share of every expense they are part of. Positive means they get money back.
- The payer does not have to be part of the split. Someone can buy groceries for the others.
- Splitting never loses or invents a paisa. ₹100 split equally between 3 people is ₹33.34 + ₹33.33 + ₹33.33. Pick a rule for who gets the extra paisa, use it everywhere, and state it in your README.
- Balances across all members always add up to exactly ₹0.
- Every amount is shown in Indian format, for example ₹1,00,000.00.

Data

There is no backend to set up. Copy the sample data at the end of this section into your project, for example as `src/data/sample-data.json`, and start the app from it.

Changes must survive a page reload, so save the expenses in `localStorage`. If nothing is saved yet, or the saved data cannot be read, start from the sample data.

A member is `{ "id": string, "name": string }`. An expense has these fields:

Field	JSON type	Notes
`id`	string	Created by your app when an expense is added
`description`	string	
`amount`	integer	In paise: ₹1,250.50 is `125050`

Field	JSON type	Notes
`paidBy`	string	A member id
`date`	string	`YYYY-MM-DD`
`category`	string	`Food`, `Travel`, `Stay`, `Activities` or `Other`
`split`	object	One of the three shapes below, told apart by `type`

- Equally: `{ "type": "equal", "memberIds": ["m1", "m2"] }`
- Exact amounts: `{ "type": "exact", "shares": { "m1": 50000, "m2": 25000 } }`, in paise per member
- Percentages: `{ "type": "percent", "shares": { "m1": 60, "m2": 40 } }`, in percent per member

You write the TypeScript types for all of this yourself.

Sample data

```
{
  "members": [
    { "id": "m1", "name": "Aarav" },
    { "id": "m2", "name": "Diya" },
    { "id": "m3", "name": "Kabir" },
    { "id": "m4", "name": "Meera" },
    { "id": "m5", "name": "Rohan" }
  ],
  "expenses": [
    {
      "id": "e1",
      "description": "Villa booking",
      "amount": 2400000,
      "paidBy": "m1",
      "date": "2026-03-12",
      "category": "Stay",
      "split": { "type": "equal", "memberIds": ["m1", "m2", "m3", "m4", "m5"] }
    },
    {
      "id": "e2",
      "description": "Beach shack dinner",
      "amount": 325000,
      "paidBy": "m2",
      "date": "2026-03-12",
      "category": "Food",
      "split": { "type": "equal", "memberIds": ["m1", "m2", "m3", "m4"] }
    },
    {
      "id": "e3",
      "description": "Scuba diving",
      "amount": 1050000,
      "paidBy": "m3",
```

```
      "date": "2026-03-13",
      "category": "Activities",
      "split": { "type": "exact", "shares": { "m3": 350000, "m4": 350000, "m5":
350000 } }
    },
    {
      "id": "e4",
      "description": "Scooter rental",
      "amount": 200000,
      "paidBy": "m5",
      "date": "2026-03-13",
      "category": "Travel",
      "split": { "type": "percent", "shares": { "m5": 50, "m1": 25, "m2": 25 } }
    },
    {
      "id": "e5",
      "description": "Groceries",
      "amount": 100000,
      "paidBy": "m4",
      "date": "2026-03-14",
      "category": "Food",
      "split": { "type": "equal", "memberIds": ["m1", "m2", "m3"] }
    },
    {
      "id": "e6",
      "description": "Airport cab",
      "amount": 149999,
      "paidBy": "m1",
      "date": "2026-03-15",
      "category": "Travel",
      "split": { "type": "percent", "shares": { "m1": 33.33, "m2": 33.33, "m3": 33.34
} }
    }
  ]
}
```

Technical requirements

- **React and TypeScript**, in a project you scaffold yourself. Vite is the quickest. Use React Router for the pages.
- **Strict TypeScript.** `"strict": true` in `tsconfig`, and `npm run typecheck` passes with zero errors. No `any`, `@ts-ignore` or `@ts-expect-error`. Avoid `as` casts and the `!` non-null operator; if you truly need one, add a comment saying why.
- **Redux Toolkit** holds the members and expenses, with typed hooks such as `useAppDispatch` and `useAppSelector`. Where you read and write `localStorage` is your call; say why in the README.
- **Logic outside components.** Money parsing, splitting, balances and validation live in plain TypeScript functions you can test without React. Components render and handle events.
- **Tests** with Vitest or Jest and React Testing Library. At minimum: the split and balance logic, including ₹100 split equally between 3; and the form, both showing errors for invalid input and saving valid input.
- **ESLint** with typescript-eslint, and `npm run lint` passes.

- **No form or validation libraries** (such as React Hook Form, Formik, Zod or Yup) and **no component libraries** (such as MUI, Ant Design or Chakra). We want to see how you model forms and write HTML and CSS. Any styling approach is fine: plain CSS, CSS Modules, Sass or Tailwind.
- **Responsive** down to 360 px wide. On small screens the table can become cards or scroll sideways; your choice.
- **Accessible basics.** Every input has a visible label, errors appear next to their field, and the whole app works with a keyboard.

Your `package.json` should have these scripts: `dev`, `build`, `typecheck`, `lint` and `test`.

What to submit

A link to a GitHub repo, public or private with us invited. We should be able to run it with:

```
npm install
npm run dev
```

Your README, in about 500 words at most, covers:

1. How to run it, and anything that is not finished.
2. Three decisions you made and why, such as where each piece of state lives or how you structured the form.
3. What you would improve with another day.
4. How you used AI tools, in 2–3 sentences.

AI tools

Use whatever you normally use, including AI assistants. We want to see how you actually work. We ask two things:

- You understand and can explain every line you submit. We will ask about specific parts of your code.
- For the short live change in the follow-up, you drive. Editor autocomplete is fine; asking an AI to write the change is not.

Using AI is never scored down. Code you cannot explain is.

How we evaluate

We look at correct money logic, type safety, how state is split between Redux and components, how cleanly logic is separated and reused, tests that check real behavior, and a usable, responsive UI.

We do not look at visual polish, animations, number of commits, amount of code, or extra features.

The follow-up call (If the assignment shortlisted)

You walk us through your app and code for a few minutes. We ask about a few parts of it, then you make one small change while sharing your screen. The last few minutes are for your questions.